

Lo-Fi Prototyping

Ziel

Mit diesem Lo-Fi Prototyp wollen wir folgende Fragestellungen klären:

1. **Verständlichkeit:** Können Nutzer:innen komplexe politische Entscheidungen durch vereinfachte Sprache und strukturierte Darstellung besser verstehen?
2. **Navigation & Informationsarchitektur:** Welche Ansicht bevorzugen Nutzer:innen – chronologische Timeline oder thematische Gruppierung? Wie intuitiv ist der Wechsel zwischen beiden?
3. **Informationstiefe:** Fühlen sich Nutzer:innen durch die gestaffelte Information (Übersicht → Zusammenfassung → Volltranskript) gut geleitet oder ist es zu verschachtelt?
4. **Feature-Relevanz:** Werden Such- und Filterfunktionen tatsächlich genutzt? Ist die Text-to-Speech Funktion ein echter Mehrwert oder Gimmick?
5. **Engagement:** Motiviert das Design dazu, mehrere Themen zu erkunden, oder wirkt es zu nüchtern/technisch?

Umsetzung

Form: Funktionaler Klick-Dummy (interaktive Web-Anwendung)

Warum diese Form?

- Civic-Tech-Anwendungen leben von Interaktivität – statische Wireframes würden das Nutzererlebnis nicht ausreichend simulieren
- Filter-, Such- und Audio-Features müssen erlebbar sein, um sie zu bewerten
- Schnelles Iterieren möglich durch React-Komponenten
- Realistische Testumgebung für User Testing mit echten Geräten (Desktop & Mobile)
- Geringe Investition trotz hoher Interaktivität (keine Backend-Entwicklung nötig)

Screens & Hauptansichten

1. Timeline-Ansicht (Haupteinstieg)

- Chronologische Darstellung aller Ratsentscheidungen
- Status-Indikator (genehmigt/ausstehend/abgelehnt)
- Klare Hierarchie: Titel → Datum → Zusammenfassung → Entscheidung

Annahme: Nutzer:innen wollen primär wissen "Was ist gerade passiert?" und orientieren sich chronologisch.

2. Thematische Ansicht

- Gruppierung nach Kategorien (Verkehr, Wohnen, Parks, etc.)
- Visuelle Differenzierung durch Icons und Farben
- Übersicht über Aktivität pro Themenbereich

Annahme: Engagierte Bürger:innen haben spezifische Interessenschwerpunkte und wollen gezielt navigieren.

3. Such- & Filterleiste

- Keyword-Suche über alle Inhalte (inkl. Transkripte)
- Datums-Filter für zeitliche Eingrenzung
- Kombinierbar mit Themen-Filter

Annahme: Power-User brauchen präzise Recherche-Tools für spezifische Themen oder Zeiträume.

4. Detail-Ansicht

- Dreiteilige Struktur:
- "Was das bedeutet" (Plain Language Summary)
- "Auswirkung auf die Community" (Impact)
- "Diskussions-Highlights" (Transkript-Auszüge)
- Call-to-Action: "Vollständiges Transkript ansehen"

Annahme: Abgestufte Information verhindert Overwhelm – Nutzer:innen entscheiden selbst, wie tief sie einsteigen.

5. Volltranskript mit TTS

- Umschaltung zwischen Lese- und Hör-Modus
- Audio-Controls (Play/Pause/Restart)
- Visuelles Highlighting des aktuell vorgelesenen Abschnitts

Annahme: Barrierefreiheit und alternative Nutzungsformate (z.B. "Podcast-Modus" beim Pendeln) erhöhen Reichweite.

Funktionsweise & Annahmen-Überprüfung

Kern-Interaktion:

1. **Einstieg:** Nutzer:in landet auf Timeline → sieht sofort neueste Entscheidungen
2. **Exploration:** Klick auf Karte → Detail-Ansicht mit vereinfachter Erklärung
3. **Vertiefung:** Optional Transkript lesen oder anhören
4. **Alternative:** Über Themen-Filter oder Suche gezielt navigieren

Zu validierende Annahmen:

-  Spacious Design reduziert kognitive Last bei komplexen Themen
-  "Plain Language"-Ansatz macht politische Inhalte zugänglicher
-  TTS-Feature wird aktiv genutzt (oder nur "nice to have"?)
-  Nutzer:innen erkunden mehrere Themen pro Session (oder nur ein spezifisches?)
-  Balance zwischen Information und Übersichtlichkeit ist getroffen

Lernfokus für nächste Iteration

Vor Hi-Fi Prototyping müssen wir herausfinden:

1. **Content-Struktur:**

- Ist die Dreiteilung (Summary/Impact/Highlights) die richtige Granularität?
- Fehlen wichtige Informationen (z.B. Abstimmungsergebnis, beteiligte Personen)?

1. **Feature-Priorisierung:**

- Welche Features werden ignoriert? (→ entfernen oder prominenter machen)
- Welche fehlen? (z.B. Benachrichtigungen, Kommentare, Teilen-Funktion)

1. **Visuelle Hierarchie:**

- Lenkt das Design die Aufmerksamkeit auf die wichtigsten Infos?
- Ist der "Spacious"-Ansatz zu minimalistisch oder genau richtig?

1. **Nutzungskontext:**

- Wird mobil oder Desktop genutzt?
- Kurze Sessions (schnell informieren) oder lange (recherchieren)?

1. **Accessibility:**

- Funktioniert TTS in allen Browsern zuverlässig?
- Ist Kontrast/Schriftgröße für alle Altersgruppen geeignet?

1. **Content-Strategie:**

- Wie viel "Plain Language"-Vereinfachung ist nötig, ohne zu vereinfachen?
- Muss die Original-Formulierung sichtbar sein?

Prototyp Link

<https://ranch-crush-31990840.figma.site/>

Code Prototyp

Moosburg Transparent – Digitale Plattform für transparente Stadtratsentscheidungen

1. Ziel des Lo-Fi Prototyps

1.1 Zentrale Fragestellungen

Mit diesem Lo-Fi Prototyp wollen wir folgende grundlegende Fragen klären:

Usability & Verständlichkeit

- **Verstehen Bürger:innen die vereinfachten Antragsbeschreibungen?**
Können komplexe politische Entscheidungen durch einfache Sprache und Visualisierungen verständlich gemacht werden?
- **Ist die Navigation intuitiv?**
Finden Nutzer:innen schnell relevante Informationen zu Stadtratsentscheidungen in ihrer Umgebung?
- **Funktionieren die Filter-Mechanismen?**
Können Nutzer:innen effektiv nach Kategorien, Status, Stadtteilen und Suchbegriffen filtern?

Engagement & Interaktion

- **Motiviert die Plattform zur Beteiligung?**
Nutzen Bürger:innen die symbolische Abstimmungsfunktion und Kommentarfunktion?
- **Schafft Transparenz Vertrauen?**
Erhöht die detaillierte Darstellung von Abstimmungsergebnissen (wer hat wie gestimmt) das Vertrauen in politische Prozesse?

Content & Information Architecture

- **Ist die Informationstiefe angemessen?**

Bieten wir genug Kontext (Timeline, Abstimmungsdetails, Kosten), ohne zu überfordern?

- **Welche Features sind wirklich relevant?**

Welche Funktionen werden am meisten genutzt (Kartenansicht, Filter, Bürgerstimmen)?

2. Umsetzung

2.1 Gewählte Form: **Funktionaler Click-Dummy (Next.js Web-App)**

Wir haben uns für einen **funktionalen Click-Dummy als interaktive Web-Anwendung** entschieden – nicht nur Wireframes oder statische Mockups, sondern eine lauffähige Prototyp-Website.

2.2 Technologie-Stack

Framework: Next.js 16 (React)
Styling: Tailwind CSS 4
Sprache: TypeScript
Deployment: Lokal (Dev-Server)

2.3 Warum diese Form?

Vorteil	Begründung
Realistische Interaktion	Nutzer:innen können echte Klicks ausführen, filtern, suchen – nicht nur durch Bilder klicken
Responsive Design testbar	Mobile First-Ansatz kann direkt auf verschiedenen Geräten getestet werden
Schnelle Iterationen	Code-basiert ermöglicht schnelle präzise Anpassungen nach Feedback
Nähe zum Endprodukt	Technische Machbarkeit wird früh sichtbar
User Testing geeignet	Kann direkt mit Testern durchlaufen werden (z.B. über localhost oder Deployment)

2.4 Designprinzipien

- **Einfache Sprache:** Kein Amtsdeutsch, sondern verständliche Zusammenfassungen
 - **Visuelle Hierarchie:** Emojis, Farben und Icons für schnelle Orientierung
 - **Mobile First:** Optimiert für Smartphone-Nutzung
 - **Barrierefreiheit:** Große Schrift, hohe Kontraste, semantisches HTML
 - **Minimalistisch:** Focus auf wesentliche Informationen, keine Überladung
-

3. Screens & Funktionalität

3.1 Startseite (/)

Ziel: Schneller Überblick und Einstieg in die Plattform

Features:

- **Hero-Section** mit Slogan: "Verstehen, was entschieden wird"
- **Statistik-Dashboard:**
 - Anzahl Anträge gesamt
 - Angenommene Anträge
 - Anträge in Beratung
 - Gesamtzahl Bürgerstimmen
- **Nächste Stadtratssitzung:** Datum, Uhrzeit, Ort prominent angezeigt
- **Neueste Anträge:** Die 4 aktuellsten Stadtratsentscheidungen als Cards
- **Call-to-Actions:** "Alle Anträge ansehen" und "Mehr erfahren"

Kernfunktionen:

- ✓ Schneller Überblick über aktuelle politische Aktivitäten
- ✓ Direkter Zugang zu den wichtigsten Bereichen
- ✓ Motivierender Einstieg durch visuelle Statistiken

3.2 Anträge-Übersicht (/antraege)

Ziel: Durchsuchbare, filterbare Liste aller Stadtratsentscheidungen

Features:

- **Suchfunktion:** Volltextsuche in Titeln und Beschreibungen
- **4 Filter-Dimensionen:**
 - **Kategorie:** Verkehr, Umwelt, Soziales, Finanzen, Kultur, Bau
 - **Status:** Angenommen, Abgelehnt, In Beratung, Vertagt
 - **Stadtteil:** Geografische Zuordnung
 - **Freitext:** Keyword-Suche
- **Antragskarten (Cards):**
 - Kategorie-Icon mit Farbe
 - Status-Badge
 - Titel + Kurzfassung (2 Zeilen)
 - Abstimmungsbalken (Dafür/Dagegen)
 - Datum, Stadtteil, Kommentarzahl

Filter-Beispiel:

Kategorie: "  Verkehr"
Status: "  Angenommen"
Stadtteil: "Altstadt"
→ Zeigt: "Tempo 30 in der Altstadt"

Annahme zum Testen:

- Können Nutzer:innen schnell relevante Anträge finden?
 - Ist die Kombination mehrerer Filter verständlich?
-

3.3 Antrags-Detailseite (`/antrag/[id]`)

Ziel: Vollständige, verständliche Information zu einer Stadtratsentscheidung

Features:

Kopfbereich

- Kategorie-Badge mit Icon
- Status-Badge (farbcodiert)
- Titel (groß und prominent)
- Datum + Stadtratssitzung
- Stadtteil-Zuordnung

Hauptinhalt

- **Kurzfassung:** 2-3 Sätze "Worum geht's?"
- **Detaillierte Beschreibung:**
 - Was bedeutet das konkret?
 - Warum ist das wichtig?
 - Was kostet das?
- **Markdown-Formatierung:** Listen, Bold, Überschriften

Abstimmungsergebnisse

- **Gesamtergebnis:** Dafür/Dagegen/Enthaltung als Balkendiagramm
- **Parteidetails:** Aufschlüsselung nach Fraktionen
 - CSU, SPD, Grüne, FW, FDP
 - Farbcodierung entsprechend Parteifarben
 - Zahlen je Fraktion (Dafür/Dagegen/Enthaltung)

Timeline

Chronologischer Ablauf des Antrags:

-  01.09.2024 → Antrag eingereicht von Bürgerinitiative
-  20.09.2024 → Erste Beratung im Verkehrsausschuss
-  15.10.2024 → Gutachten der Verkehrsplanung liegt vor
-  15.11.2024 → Abstimmung im Stadtrat - Angenommen
-  01.01.2025 → Geplante Umsetzung

Bürgerbeteiligung

- **Symbolische Abstimmung:** "Wie würdest du abstimmen?"
 - Buttons: 👍 Dafür / 👎 Dagegen
 - Live-Update der Prozentzahlen
 - Bestätigung nach Abstimmung
- **Kommentarbereich:**
 - Nutzerkommentare mit Likes
 - Datum und Autor
 - (Im Prototyp: Statische Beispielkommentare)

Annahme zum Testen:

- Ist die Informationstiefe angemessen?
- Verstehen Nutzer:innen die Abstimmungsvisualisierung?
- Motiviert die Bürgerstimmenfunktion zur Interaktion?

3.4 Kartenansicht (/karte)

Ziel: Geografische Verortung von Stadtratsentscheidungen

Features (geplant):

- Interaktive Karte von Moosburg
- Marker für jeden Antrag mit Stadtteil-Zuordnung
- Klick auf Marker → Antrag-Details

- Filter auch in Kartenansicht verfügbar

Status im Lo-Fi Prototyp: Placeholder-Seite

- Zeigt Konzept, aber noch keine echte Karten-Integration
 - Begründung: Fokus liegt zunächst auf Informationsarchitektur und Verständlichkeit
-

3.5 Über Uns (/ueber-uns)

Ziel: Transparenz über die Plattform selbst

Features:

- **Mission Statement:** Warum existiert die Plattform?
 - **Was wir bieten:** 6 Kernfunktionen mit Icons
 - Einfache Sprache
 - Visualisierte Abstimmungen
 - Interaktive Karte
 - Bürgerbeteiligung
 - Mobile First
 - Barrierefreiheit
 - **Zielgruppen:** Wer profitiert von der Plattform?
 - **Kontaktmöglichkeit** (im Prototyp: statisch)
-

4. Technische Erläuterung

4.1 Datenstruktur

Antrag-Interface (TypeScript)

```
interface Antrag {  
  id: string;  
  titel: string;
```

```

kurzfassung: string;
beschreibung: string; // Markdown-Support
kategorie: 'verkehr' | 'umwelt' | 'soziales' | 'finanzen' | 'kultur' | 'bau';
status: 'angenommen' | 'abgelehnt' | 'in-beratung' | 'vertagt';
datum: string;
sitzung: string;
stadtteil?: string;
abstimmung: {
  dafuer: number;
  dagegen: number;
  enthaltung: number;
  details: Abstimmung[]; // Partei-Details
};
timeline: { datum: string; ereignis: string; }[];
kommentare: { id, autor, text, datum, likes }[];
buergerAbstimmung: { dafuer: number; dagegen: number; };
}

```

Datenhaltung im Prototyp

- **Statische Daten:** `src/data/antraege.ts`
- 6 Beispiel-Anträge mit realistischen Daten
- Kategorien: Verkehr, Umwelt, Soziales, Finanzen, Kultur, Bau
- Status-Verteilung: Angenommen, Abgelehnt, In Beratung, Vertagt

Warum statisch?

- Fokus auf UX, nicht Backend
- Schnelle Iteration ohne Datenbank-Setup
- Ausreichend für User Testing

4.2 Komponenten-Architektur

```

src/
├── app/           # Next.js App Router

```

```

|   |— page.tsx      # Startseite
|   |— antraege/page.tsx # Antragsübersicht
|   |— antrag/[id]/page.tsx # Detailseite
|   |— karte/page.tsx   # Kartenansicht
|   |— ueber-uns/page.tsx # Über uns
|
|— components/          # Wiederverwendbare UI-Komponenten
|   |— AntragCard.tsx   # Card-Komponente für Übersicht
|   |— BuergerAbstimmung.tsx # Interaktive Abstimmung
|   |— VotingChart.tsx  # Visualisierung der Abstimmungen
|   |— Kommentare.tsx   # Kommentar-Sektion
|   |— Header.tsx       # Navigation
|   |— Footer.tsx       # Footer
|
|— data/
|   |— antraege.ts      # Mock-Daten

```

4.3 Styling-System

CSS Custom Properties (Design Tokens):

```

--primary: #2C5F7C      /* Hauptfarbe (Blau) */
--primary-dark: #1E4A5F /* Dunkleres Blau */
--secondary: #E8F4F8    /* Heller Hintergrund */
--accent: #FF6B35       /* Akzentfarbe (Orange) */
--success: #4CAF50      /* Grün (Angenommen) */
--danger: #F44336       /* Rot (Abgelehnt) */
--warning: #FFC107      /* Gelb (In Beratung) */
--neutral-gray: #666666 /* Textfarbe */

```

Tailwind CSS für Responsive Design:

- Mobile First-Ansatz
- Breakpoints: sm, md, lg, xl
- Utility-First-Classes für schnelle Anpassungen

--

5. Überprüfte Annahmen

5.1 Hypothesen, die wir testen

Hypothese	Messgröße im Prototyp	Erwartetes Ergebnis
Nutzer:innen verstehen vereinfachte Anträge besser als Originaltext	Feedback-Fragen nach User Testing	>80% finden Beschreibungen verständlich
Filter-Funktionen werden aktiv genutzt	Beobachtung während User Testing	Nutzer:innen filtern nach Kategorie/Stadtteil
Visualisierte Abstimmungen erhöhen Transparenz	Fragebogen nach Demo	>75% empfinden Abstimmungsdetails als hilfreich
Bürgerstimmen-Feature motiviert zur Interaktion	Click-Tracking (später)	>50% der Nutzer:innen stimmen ab
Mobile Nutzung ist primär	Analytics (nach Deployment)	>60% mobile Sessions

5.2 Bewusste Design-Entscheidungen

Emojis statt Icons

- **Annahme:** Emojis sind universell verständlich und lockern das UI auf
- **Zu testen:** Wirkt das unseriös? Oder zugänglicher?

Symbolische Bürgerstimmen

- **Annahme:** Auch ohne rechtliche Verbindlichkeit wollen Bürger:innen ihre Meinung äußern
- **Zu testen:** Wird die Funktion genutzt? Oder als nutzlos empfunden?

Detaillierte Abstimmungsergebnisse

- **Annahme:** Bürger:innen wollen wissen, wie ihre Vertreter:innen abstimmen
- **Zu testen:** Ist die Darstellung nach Parteien zu komplex?

Timeline-Funktion

- **Annahme:** Der Weg einer Entscheidung ist für Nutzer:innen relevant
 - **Zu testen:** Wird die Timeline gelesen? Oder übersprungen?
-

6. Lernfokus: Was müssen wir vor dem Hi-Fi Prototyp wissen?

6.1 Kritische Erfolgsfaktoren

Bevor wir in die detaillierte Hi-Fi-Phase gehen, müssen wir folgende Fragen klären:

User Experience

1. Verständlichkeit:

- Sind die vereinfachten Beschreibungen wirklich besser als Originaltexte?
- Welche Informationstiefe ist optimal? (Zu viel/zu wenig Detail?)

2. Navigation:

- Finden Nutzer:innen schnell, was sie suchen?
- Sind 4 Filter zu viel? Zu wenig?
- Soll es eine Suchvorschläge-Funktion geben?

3. Engagement:

- Nutzen Bürger:innen die Abstimmungsfunktion?
- Werden Kommentare gelesen?
- Motiviert die Plattform zur Wiederkehr?

Content-Strategie

1. Informationsarchitektur:

- Sind die Kategorien (Verkehr, Umwelt, etc.) intuitiv?
- Fehlen wichtige Informationen auf der Detailseite?

- Ist die Timeline relevant?

2. Visualisierungen:

- Verstehen Nutzer:innen die Abstimmungsdiagramme?
- Helfen die Farben oder verwirren sie?
- Sind Emojis hilfreich oder störend?

Technische Features

1. Priorisierung:

- Welche Features werden wirklich genutzt?
- Was kann in V1 weggelassen werden?
- Ist die Kartenansicht ein Must-Have oder Nice-to-Have?

2. Performance:

- Lädt die Seite schnell genug auf Mobile?
- Sind Animationen sinnvoll oder ablenkend?

7. - Entfernt

8. Fazit & Reflexion

8.1 Was haben wir erreicht?

- ✓ **Funktionaler Prototyp:** Keine statischen Bilder, sondern echte Interaktivität
- ✓ **Realistische Daten:** 6 Beispiel-Anträge mit echten Moosburger Themen
- ✓ **Vollständige User Journey:** Von Startseite über Suche bis zur Abstimmung
- ✓ **Mobile-optimiert:** Responsive Design bereits implementiert
- ✓ **Testbar:** Kann direkt mit Nutzer:innen durchlaufen werden

8.2 Limitierungen des Lo-Fi Prototyps

- ⚠ **Keine echten Daten:** Noch keine Anbindung an offizielle Stadtratsprotokolle
- ⚠ **Statische Kommentare:** Keine echte Nutzer-Interaktion

⚠ **Kartenansicht Placeholder:** Noch keine Geo-Integration

⚠ **Kein Backend:** Alle Daten im Frontend hardcoded

8.3 Lessons Learned

- **Code-basierter Prototyp lohnt sich:** Mehr Aufwand initial, aber bessere Testbarkeit
- **Mobile First ist essenziell:** Stadtratsinformationen werden unterwegs gesucht
- **Visualisierung ist Schlüssel:** Diagramme kommunizieren schneller als Text

Repo Link

https://github.com/MoritzFinkel/MVP_Service_Design